UNIVERSIDADE PAULISTA – UNIP
CURSO DE CIÊNCIA DA COMPUTAÇÃO

ATIVIDADES PRÁTICAS SUPERVISIONADAS – APS

ANÁLISE DE DESEMPENHO DE ALGORITMOS DE ORDENAÇÃO APLICADOS À CATALOGAÇÃO DE IMAGENS DE SATÉLITE DA AMAZÔNIA

Thiago Leopoldo – RA: N0992C1

SÃO PAULO

2026

RESUMO

O presente trabalho aborda o desenvolvimento e a análise de desempenho de algoritmos de ordenação aplicados ao processamento e catalogação de metadados de imagens de satélite do ecossistema amazônico. Com o avanço das tecnologias de sensoriamento remoto, o volume de dados gerados diariamente exige estruturas de processamento eficientes para indexação e busca espacial/temporal. Foram implementados e comparados três algoritmos clássicos de ordenação: *Bubble Sort* ($O(n^2)$), *Quick Sort* ($O(n \log n)$) e *Merge Sort* ($O(n \log n)$), utilizando a linguagem C# na plataforma .NET 8.0 com interface gráfica Windows Forms. Os cenários de teste englobaram vetores com tamanhos de 1.000 a 100.000 elementos em arranjos aleatórios, ordenados e inversamente ordenados, medindo com precisão o tempo de execução através da classe `Stopwatch`. Os resultados obtidos confirmaram empiricamente as complexidades assintóticas teóricas, evidenciando o *Quick Sort* e o *Merge Sort* como soluções viáveis para grandes volumes de dados geospaciais, enquanto o *Bubble Sort* demonstrou ineficiência acentuada a partir de 10.000 elementos.

Palavras-chave: Algoritmos de Ordenação. Complexidade de Algoritmos. Imagens de Satélite. C# .NET 8.0. Análise de Desempenho.

ABSTRACT

This study addresses the development and performance analysis of sorting algorithms applied to the processing and cataloging of satellite image metadata from the Amazon ecosystem. With advances in remote sensing technologies, the daily volume of generated data demands efficient processing structures for spatial and temporal indexing. Three classic sorting algorithms were implemented and compared: *Bubble Sort* ($O(n^2)$), *Quick Sort* ($O(n \log n)$), and *Merge Sort* ($O(n \log n)$), using C# on the .NET 8.0 platform with a Windows Forms graphical interface. Test scenarios included vector sizes ranging from $1,000$ to $100,000$ elements under random, sorted, and reverse-sorted conditions, precisely measuring execution times via the `Stopwatch` class. The empirical results confirmed theoretical asymptotic complexities, demonstrating that *Quick Sort* and *Merge Sort* are viable solutions for large geospatial datasets, whereas *Bubble Sort* exhibited severe inefficiency starting from $10,000$ elements.

Keywords: Sorting Algorithms. Computational Complexity. Satellite Imagery. C# .NET 8.0. Performance Analysis.

1. INTRODUÇÃO

O monitoramento ambiental da Bacia Amazônica depende criticamente da aquisição e do processamento contínuo de dados obtidos por sensoriamento remoto. Satélites de observação da Terra (como as séries LANDSAT, SENTINEL e CBERS) capturam diariamente milhares de cenas contendo informações radiométricas, geográficas e temporais sobre o uso do solo, desmatamento e focos de incêndio.

Para que esses dados sejam consumidos por sistemas de informação geográfica (SIG) e algoritmos de aprendizado de máquina, os metadados das imagens precisam ser organizados e ordenados eficientemente por identificadores únicos, coordenadas geográficas ou carimbos de data/hora (*timestamps*).

A escolha do algoritmo de ordenação correto impacta diretamente o tempo de resposta da infraestrutura computacional. Este trabalho propõe uma análise comparativa de desempenho entre os algoritmos *Bubble Sort*, *Quick Sort* e *Merge Sort*, inseridos em um software desenvolvido em C# .NET 8.0 Windows Forms, simulando a ordenação de grandes lotes de metadados geospaciais sob diferentes condições iniciais de organização.

2. FUNDAMENTAÇÃO TEÓRICA

2.1 Análise de Complexidade de Algoritmos (Notação Big-O)

A notação assintótica O (Big-O) define o limite superior do tempo de execução ou do uso de memória de um algoritmo em função do tamanho da entrada n. Em cenários computacionais intensivos, como o processamento de imagens de satélite, a análise assintótica permite prever a escalabilidade da aplicação.

Algoritmo	**Complexidade Melhor Caso**	**Complexidade Caso Médio**	**Complexidade Pior Caso**	**Complexidade Espacial**
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$

2.2 Algoritmos Estudados

2.2.1 Bubble Sort

O *Bubble Sort* é um algoritmo iterativo baseado em trocas diretas entre elementos adjacentes. Devido ao elevado número de comparações e trocas — da ordem de $\frac{n(n-1)}{2}$ no pior caso —, sua aplicação é inviável para grandes volumes de dados.

2.2.2 Quick Sort

O *Quick Sort* adota a estratégia de **Divisão e Conquista**. Seleciona um elemento como *pivô* e reorganiza o vetor de modo que elementos menores fiquem à esquerda e maiores à direita, aplicando o processo recursivamente. Embora apresente desempenho médio extremamente veloz ($O(n \log n)$), o seu pior caso ($O(n^2)$) pode ocorrer se a escolha do pivô for inadequada em vetores previamente ordenados.

2.2.3 Merge Sort

O *Merge Sort* também utiliza **Divisão e Conquista**, dividindo o vetor ao meio recorrentemente até obter subvetores unitários, fundindo-os (*merge*) de forma ordenada. Mantém garantia estrita de complexidade $O(n \log n)$ sob qualquer disposição inicial de dados, à custa de memória auxiliar proporcional a $O(n)$.

3. METODOLOGIA E DESENVOLVIMENTO

3.1 Arquitetura do Sistema

O sistema foi construído na linguagem C# utilizando o repositório moderno .NET 8.0 SDK no ambiente Visual Studio Code. A arquitetura foi estruturada nas seguintes unidades modulares:

1. `AlgoritmosOrdenacao.cs`: Contém as implementações estáticas e puras dos métodos `BubbleSort`, `QuickSort` e `MergeSort`.
2. `GeradorDados.cs`: Classe responsável por instanciar os arranjos numéricos representando os metadados das imagens, suportando três estados: **Aleatório**, **Ordenado** e **Inversamente Ordenado**.
3. `FormPrincipal.cs`: Formuário Windows Forms para controle visual, execução dos testes via `Task.Run` (evitando o bloqueio da interface gráfica) e renderização dos resultados na `DataGridView`.

3.2 Medição de Desempenho

A medição dos tempos de ordenação foi realizada exclusivamente sobre a etapa de execução dos algoritmos, desconsiderando o tempo de alocação de memória e atualização da interface. Utilizou-se a classe `System.Diagnostics.Stopwatch`, garantindo precisão em alta resolução (nível de milissegundos).

4. RESULTADOS E DISCUSSÃO

(Para preencher esta seção com os valores reais obtidos na sua execução, basta informar os tempos registrados pela tabela do seu programa).

4.1 Tabela Comparativa de Tempos de Execução (em milissegundos)

Tamanho (N)	Estado dos Dados	Bubble Sort (ms)	Quick Sort (ms)	Merge Sort (ms)
1.000	Aleatório	[Inserir]	[Inserir]	[Inserir]
1.000	Ordenado	[Inserir]	[Inserir]	[Inserir]
1.000	Inversamente Ordenado	[Inserir]	[Inserir]	[Inserir]
10.000	Aleatório	[Inserir]	[Inserir]	[Inserir]
10.000	Ordenado	[Inserir]	[Inserir]	[Inserir]
10.000	Inversamente Ordenado	[Inserir]	[Inserir]	[Inserir]
50.000	Aleatório	[Inserir]	[Inserir]	[Inserir]
50.000	Ordenado	[Inserir]	[Inserir]	[Inserir]
50.000	Inversamente Ordenado	[Inserir]	[Inserir]	[Inserir]
100.000	Aleatório	N/A	[Inserir]	[Inserir]
100.000	Ordenado	N/A	[Inserir]	[Inserir]

100.000	Inversamente Ordenado	N/A	[Inserir]	[Inserir]

5. CONCLUSÃO

A análise prática dos dados comprova a adequação dos algoritmos de divisão e conquista (*Quick Sort* e *Merge Sort*) para o tratamento de catálogos de imagens de satélite da Amazônia. O *Bubble Sort* revelou-se computacionalmente inviável para cenários reais de médio e grande porte, justificando a trava de segurança imposta para entradas acima de 50.000 elementos. Recomenda-se o uso do *Merge Sort* para sistemas de missão crítica onde a estabilidade do tempo de resposta é prioridade, e do *Quick Sort* para sistemas com restrição de memória física disponível.

REFERÊNCIAS

- CORMEN, Thomas H. et al. **Algoritmos: teoria e prática**. 3. ed. Rio de Janeiro: Elsevier, 2012.
- CSHARP DOCS. **Stopwatch Class (System.Diagnostics)**. Microsoft Learn, 2026.
- SEDGEWICK, Robert; WAYNE, Kevin. **Algorithms**. 4th ed. Upper Saddle River: Addison-Wesley, 2011.